UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

FACULTAD DE INGENIERÍA EN SISTEMAS

CENTRO UNIVERSITARIO SAN JOSÉ PINULA

CURSO: PROGRAMACIÓN II – SECCIÓN: “C”

CATEDRÁTICO: ING. DANILO ESCOLIN

GUÍA DE INSTALACIÓN Y DESPLIEGUE DE CITANOVA

MORALES MONTERROSO ANGEL GABRIEL – 3190-25-9912

ARIAS CHICAS AMERICA ALESSANDRA – 3190-25-5550

NATHAN GADIEL PIEDRASANTA VIELMAN – 3190-25-7278

JULIÁN ALEJANDRO ROCA GUERRA – 3190-25-19627

JAIME DANILO VELÁSQUEZ MARTÍNEZ – 3190-25-16540

ANGEL ISAAC GONZÁLEZ RAMÍREZ – 3190-25-75

Pasos para ejecutar CitaNova en Windows o Linux, usando MariaDB o MySQL.

1. Requisitos

Instala Git, Java JDK 21, Node.js 24.15.0 o superior de la rama 24 y una base de datos: MariaDB o MySQL 8.4. El proyecto incluye Maven Wrapper.

Windows: usa los instaladores, agrega las herramientas al PATH e inicia la base desde Servicios. Ejecuta los comandos en PowerShell.

Linux (Ubuntu/Debian): inicia la base con `sudo systemctl start mariadb` o `sudo systemctl start mysql`. Ejecuta los comandos en Bash.

2. Base de datos

Desde la carpeta principal del proyecto, abre el cliente correspondiente:

- MariaDB: `mariadb --default-character-set=utf8mb4 -u root -p`.
- MySQL: `mysql --default-character-set=utf8mb4 -u root -p`.

En Linux, si root autentica mediante el sistema, usa `sudo mariadb --default-character-set=utf8mb4` o `sudo mysql --default-character-set=utf8mb4`. Dentro del cliente, importa el esquema y crea el usuario y los estados; cambia la contraseña:

```
SOURCE citas-backend/database/schema.sql;
CREATE USER 'citanova'@'127.0.0.1' IDENTIFIED BY 'CAMBIAR_PASSWORD';
GRANT ALL PRIVILEGES ON citas_db.* TO 'citanova'@'127.0.0.1';
USE citas_db;
INSERT IGNORE INTO estados_cita (nombre, descripcion) VALUES
  ('PENDIENTE', 'Cita pendiente de atención'),
  ('COMPLETADA', 'Cita atendida'),
  ('CANCELADA', 'Cita cancelada');
```

Estos pasos SQL son iguales en ambos sistemas y motores.

3. Backend

En `citas-backend/src/main/resources/application.properties`, configura:

```
spring.datasource.url=jdbc:mariadb://127.0.0.1:3306/citas_db
spring.datasource.driver-class-name=org.mariadb.jdbc.Driver
spring.datasource.username=citanova
spring.datasource.password=CAMBIAR_PASSWORD
jwt.secret=CAMBIAR_POR_UN_SECRETO_ALEATORIO_DE_AL_MENOS_32_BYTES
```

El ejemplo usa MariaDB. Para MySQL, activa en `citas-backend/pom.xml` la dependencia comentada `com.mysql:mysql-connector-j` y sustituye estas dos propiedades:

```
spring.datasource.url=jdbc:mysql://127.0.0.1:3306/citas_db
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
```

Usa la contraseña creada y una clave JWT propia. Desde `citas-backend`, inicia con `./mvnw spring-boot:run` en Linux o `.\mvnw.cmd spring-boot:run` en Windows. En Linux, si falta permiso, ejecuta `chmod +x mvnw`.

Mantén la terminal abierta. Comprueba el backend en `http://localhost:8081/swagger-ui/index.html`.

Para correos locales, instala Mailpit. En otra terminal, ejecuta en cualquiera de los sistemas:

```
mailpit --listen 127.0.0.1:8025 --smtp 127.0.0.1:1025
```

Consulta los mensajes en `http://localhost:8025`.

4. Aplicación web

Los pasos web son iguales en ambos sistemas. Si PowerShell bloquea scripts, usa `npm.cmd` y `npx.cmd` en lugar de `npm` y `npx`. Abre otra terminal en la carpeta principal:

```
cd citas-frontend
npm ci
npm start
```

Entra a `http://localhost:4200`. Si cambias el puerto 8081 del backend, actualiza `proxy.conf.json`.

Registra una cuenta en `/register`. Para convertirla en administradora, ejecuta este SQL en MariaDB o MySQL, sustituyendo el correo, y vuelve a iniciar sesión:

```
USE citas_db;
INSERT IGNORE INTO usuario_rol (id_usuario, id_rol)
SELECT u.id_usuario, r.id_rol
FROM usuarios u JOIN roles r ON r.nombre = 'ROLE_ADMIN'
WHERE u.correo = 'CORREO_DEL_ADMINISTRADOR';
```

5. Aplicación móvil

Desde la carpeta principal, tanto en Windows como en Linux:

```
cd citas-ionic
npm ci
```

Para probar en el navegador, cambia `apiUrl` a `/api/v1` en `src/environments/environment.ts`, ejecuta `npm start` y abre `http://localhost:8100`.

Para Android, instala Android Studio con SDK 36 y JDK 21. En `src/environments/environment.prod.ts`, configura `apiUrl`:

- Emulador: `http://10.0.2.2:8081/api/v1`.
- Teléfono en la misma red: `http://IP_DEL_EQUIPO:8081/api/v1`.
- Aplicación publicada: `https://TU_DOMINIO/api/v1`.

Desde `citas-ionic`, compila y abre Android Studio:

```
npm run build -- --configuration production
npx cap sync android
npx cap open android
```

Selecciona el dispositivo y pulsa Run. Para un APK de prueba, desde `citas-ionic/android` ejecuta `./gradlew assembleDebug` en Linux o `.\gradlew.bat assembleDebug` en Windows. Se guarda en `app/build/outputs/apk/debug/app-debug.apk`.

En un teléfono físico, permite el puerto 8081 del equipo. Si cambia el origen del cliente, agrégalo a CORS en el backend.

6. Despliegue en servidor

El servidor necesita Java 21, MariaDB o MySQL, Nginx y un dominio con HTTPS. Prepara la base según el apartado 2.

Genera el backend desde `citas-backend`: Linux, `./mvnw clean verify`; Windows, `.\mvnw.cmd clean verify`. Genera la web desde `citas-frontend`:

```
npm ci
npm run build -- --configuration production
```

Copia `target/citas-backend-0.0.1-SNAPSHOT.jar` al servidor con el nombre `citanova.jar`. Junto al JAR, crea `backend.properties` con las cinco propiedades del apartado 3, usando la URL y el controlador del motor elegido. Añade:

```
spring.jpa.hibernate.ddl-auto=validate
app.identity.frontend-base-url=https://TU_DOMINIO
app.cors.allowed-origins=https://TU_DOMINIO,https://localhost
```

Conserva una clave JWT estable, configura el SMTP real y aplica las migraciones pendientes. Desde la carpeta del JAR, inicia el backend con el mismo comando en ambos sistemas:

```
java -jar citanova.jar --spring.config.additional-location=file:./backend.properties
```

Copia el contenido de `citas-frontend/dist/citas-fronted/browser/` al directorio web:

- Linux: `/var/www/citanova`; guarda el sitio en `/etc/nginx/conf.d/citanova.conf`.
- Windows: `C:/nginx/html/citanova`; edita el bloque `server` dentro de `http` en `C:/nginx/conf/nginx.conf`.

Usa este bloque y sustituye `RUTA_WEB` por el directorio anterior:

```
server {
    listen 80;
    server_name TU_DOMINIO;
    root RUTA_WEB;
    index index.html;
    location /api/ { proxy_pass http://127.0.0.1:8081; }
    location / { try_files $uri $uri/ /index.html; }
}
```

Linux: ejecuta `sudo nginx -t` y `sudo systemctl reload nginx`.

Windows: extrae Nginx en `C:\nginx` antes de copiar la web. Desde esa carpeta:

```
.\nginx.exe -t
Start-Process .\nginx.exe
```

Tras cambiar la configuración, usa `.\nginx.exe -s reload`. En Windows funciona como aplicación, sin servicio nativo.

Habilita HTTPS y configura el arranque automático del backend según el sistema. Para distribuir Android, usa la API HTTPS, recompila y genera una versión firmada en Android Studio.

7. Comprobación final

Abre la web, inicia sesión y reserva una cita. Confirma que aparece en el historial del paciente y en la agenda del médico. Revisa los correos y la conexión móvil. Antes de actualizar, respalda la base y la configuración.